

1. Algoritmo Greedy para encontrar minimo tiempo

En este trabajo realizaremos el analisis teorico y empirico del algoritmo Greedy propuesto para resolver el problema de scheduling de videos.

1.1. Algoritmo Greedy

A continuacion se muestra el codigo de solucion Greedy del problema.

```

1 def tiemposminimos(n_equipos, scaloni, ayudantes):
2     equipos=[]
3     for i in range(1,n_equipos+1):
4         equipos.append(i)
5     aux=list(zip(scaloni,ayudantes,equipos))
6     aux=sorted(aux,key=lambda x:x[1],reverse=True)
7     tiempo_total=0
8     tiempo_ite=0
9     orden=[]
10    for (scaloni,ayudantes,equipo) in aux:
11        tiempo_ite+=scaloni
12        tiempo_total=max(tiempo_total,tiempo_ite+ayudantes)
13        orden.append(equipo)
14    return tiempo_total,orden

```

1.2. Regla Greedy

1.3. Solución Optima

Demostración por inversiones:

Tenemos 2 elementos en la solución óptima tales que $a_i < a_j$ y $i < j$, es decir, que un elemento anterior tiene un tiempo de ayudante menor al posterior. Esto va en contra de nuestro algoritmo, por lo que se necesitaría una inversión para llegar a la solución optima propuesta por nuestro algoritmo.

El orden, al no tener inversiones, implica que solo varían posiciones de videos consecutivos cuyo tiempo de analisis de ayudante sea igual. Y tiempo total maximo dado por el ultimo elemento consecutivo.

Supongo que un orden con inversiones es optimo, para que esto sea mas que una suposición tengo que descartar un problema, si al cambiar el orden de videos consecutivos el tiempo total de la solución aumenta:

T = tiempo acumulado (hasta ahora)

s =scaloni

a =ayudantes

Pre inversion: $i- > j$

$$tiempo_j = T + s_i + s_j$$

$$Totaltiempo1 = \max(T + s_i + a_i, T + s_i + s_j + a_j)$$

Post inversion: $j- > i$

$$tiempo_i = T + s_j + s_i$$

$$Totaltiempo2 = \max(T + s_j + a_j, T + s_j + s_i + a_i)$$

Teniendo en cuenta $a_i < a_j$ se cumple:

$$T + s_i + s_j + a_j > T + s_j + a_j$$

$$T + s_i + s_j + a_j > T + s_i + s_j + a_i$$

Por lo que:

$$Totaltiempo1 > Totaltiempo2$$

El tiempo máximo total de i no puede ser mayor que el tiempo máximo de total j . Demostramos que la solución no empeora, es óptima ya que no puede ser peor que el tiempo total del problema.

1.4. Complejidad Algoritmo

1.5. Análisis tiempos

Proponemos tres casos:

1.5.1. $a_i \gg s_i$

En este caso, se puede observar mediante el ejemplo de ejecución ayudantes mayor.txt que el tiempo total obtenido va a basarse en la suma del tiempo de scaloni + tiempo de ayudante del primer video post ordenamiento, es decir por el tiempo de ayudante máximo.

1.5.2. $a_i \approx s_i$

Si la variación de tiempo es pequeña, es decir que los tiempos son similares, el tiempo mínimo obligatorio obtenido va a depender exclusivamente de la suma total de tiempos de scaloni y el tiempo de ayudante del último video. Tenemos el ejemplo de ejecución tiempos similares.txt donde se corrobora el análisis del tiempo óptimo.

1.5.3. $s_i \gg a_i$

El tiempo total va a ser dominado por la suma total de tiempos de scaloni, el tiempo de ayudantes es descartable, solo se tiene en cuenta el tiempo del último video. Es igual a la variación de tiempo pequeña anteriormente explicada. El ejemplo de ejecución scaloni mayor.txt nos da una solución en línea con el análisis.

1.5.4. $s_i = a_i$

Llegamos a la misma conclusión que los 2 análisis previos, el tiempo total va a estar dictado por la suma total de tiempos de scaloni y el tiempo de ayudantes del último video. Ejemplo de ejecución tiempos iguales.txt nos verifica esta solución.